Phase 2 — Grade 6: Classification

Detailed Revision Guide — full formula breakdowns, edge cases, and common-mistake debugging

1. Why Linear Regression Can't Do Classification

Classification needs a bounded PROBABILITY (0 to 1). Linear regression's y-hat=mx+b produces ANY real number (-50, 1000, etc.) with no way to enforce that boundary. The sigmoid function fixes this by squashing any real number into the 0-1 range.

The Sigmoid Formula

`sigma(z) = 1 / (1 + e^-z) where z = mx + b (the familiar linear regression output)`

`Large positive z -> e^-z tiny -> sigma(z) approaches 1`

`Large negative z -> e^-z huge -> sigma(z) approaches 0`

`z = 0 -> sigma(0) = 1/(1+1) = 0.5 (maximum uncertainty, exactly in the middle)`

EDGE CASE: What is e, concretely?
e (Euler's number) ≈ 2.71828 -- a special constant, similar in spirit to pi, that shows up naturally in continuous growth/decay processes. For calculations here, treat it exactly like any other constant: e^{-1} = 1/e ≈ 1/2.71828 ≈ 0.3679 (negative exponent = 1 divided by). In code: np.exp(-1).

Worked Example: sigma(1)

`sigma(1) = 1/(1+e^-1) = 1/(1+0.3679) = 1/1.3679 ≈ 0.7310`

`Interpretation: about 73% probability of being class 1.`

TIP: Callback to Probability (Grade 3)
The sigmoid's output genuinely IS a probability -- specifically P(class=1 | x). Classification doesn't output a hard yes/no directly; it outputs a probability estimate, and a separate rule (the threshold) converts that into a decision.

2. Decision Threshold and Decision Boundary

The Rule

`If sigma(z) ≥ threshold (commonly 0.5) -> predict class 1`

`If sigma(z) < threshold -> predict class 0`

`Since sigma(z)=0.5 exactly when z=0, the DECISION BOUNDARY is simply wherever mx+b = 0 -- the same linear formula as before, now interpreted as the dividing line between the two predicted classes.`

EDGE CASE: Changing the threshold shifts the boundary, trading off false positives vs false negatives (flagged, not yet covered in depth)
A threshold other than 0.5 (e.g. 0.3 or 0.8) changes which predictions flip to class 1 vs class 0 -- lowering the threshold makes the model predict class 1 MORE often (catching more true positives, but also more false positives). This trade-off connects directly to precision/recall, covered in a later Evaluation Metrics grade.

3. Log Loss (Cross-Entropy Loss)

MSE doesn't work well combined with the sigmoid (it can create a bumpy, non-convex error surface that traps gradient descent). Log Loss is the loss function built specifically for probability-based classification outputs.

The Full Formula (plain text, no symbols hidden)

```
L = -1 * [ y * log(p) + (1-y) * log(1-p) ]
```

```
y = the ACTUAL label (always exactly 0 or 1). p = the model's PREDICTED probability.
```

```
The -1 sits OUTSIDE the brackets from the very start -- it is applied LAST, after whatever is inside the brackets has been simplified.
```

Case A derivation: substituting y=1

```
y * log(p) becomes 1 * log(p) = log(p)
```

```
(1-y) * log(1-p) becomes (1-1) * log(1-p) = 0 * log(1-p) = 0 (multiplying by 0 erases this ENTIRE term, regardless of what log(1-p) equals)
```

```
Add the two pieces inside the brackets: log(p) + 0 = log(p)
```

```
Apply the outside -1 (which was always there, untouched by the substitution):
```

```
L = -1 * [log(p)] = -1 * log(p) = -log(p)
```

Case B derivation: substituting y=0

```
y * log(p) becomes 0 * log(p) = 0 (this term vanishes instead)
```

```
(1-y) * log(1-p) becomes (1-0) * log(1-p) = 1 * log(1-p) = log(1-p)
```

```
Add the two pieces inside the brackets: 0 + log(1-p) = log(1-p)
```

```
Apply the outside -1: L = -1 * [log(1-p)] = -log(1-p)
```

COMMON MISTAKE: Losing track of the minus sign during substitution
It's easy to assume the minus sign 'appears' only in the final answer, as if it came from somewhere during simplification. In reality, the -1 was present in the ORIGINAL formula from the very first line, sitting OUTSIDE the square brackets the entire time. Substituting y=1 or y=0 only changes what's INSIDE the brackets; the outside -1 is untouched and gets applied as the LAST step, not discovered partway through.

COMMON MISTAKE: Using the wrong formula's variable (p vs 1-p) because of which y-case applies
A real example of the confusion: in a y=1 example with p=0.1, it looked like '1-p' should have been used (since 1-p appears somewhere in the full formula). But the y=1 CASE specifically simplifies down to using p DIRECTLY (-log(p)) -- the '1-p' term only survives in the y=0 case. Always check which case (y=1 or y=0) applies FIRST, then use ONLY that case's simplified formula -- never mix pieces from both.

Worked Examples, both cases

`Case A (y=1), p=0.7310: L = -log(0.7310) ≈ -(-0.3133) = 0.3133 (small loss, good guess)`

`Case A (y=1), p=0.1: L = -log(0.1) ≈ -(-2.3026) = 2.3026 (large loss, confident wrong)`

`Case B (y=0), p=0.1: 1-p=0.9. L = -log(0.9) ≈ -(-0.105) = 0.105 (small loss, good guess)`

`Case B (y=0), p=0.95: 1-p=0.05. L = -log(0.05) ≈ -(-3.00) = 3.00 (large loss, confident wrong)`

TIP: WHY log specifically -- the asymmetric punishment property
log of a number close to 1 is close to 0 (tiny loss). log of a number close to 0 shoots toward negative infinity (which the leading -1 flips to a huge POSITIVE loss). Since the loss always looks at the probability assigned to the TRUE class, a confident WRONG guess pushes that probability toward 0 -- straight into the steep, explosive part of the curve. A cautious wrong guess only moves it moderately low, staying on the gentle part of the curve -- producing a much smaller penalty for the same 'wrong' outcome.

EDGE CASE: log(0) is undefined -- p must never be exactly 0 or exactly 1 in practice
If p=0 exactly (or p=1 exactly, for the other case), the formula requires log(0), which is undefined (negative infinity). Real implementations clip p to something like [0.0000001, 0.9999999] to avoid this computational issue -- a practical detail worth knowing even though it wasn't needed in any worked example here.

4. Full Debugging Checklist for This Grade

---

1. Am I checking the SIGN of z first (positive -> toward 1, negative -> toward 0) before computing an exact sigmoid value, as a sanity check? 2. For Log Loss, have I identified which case (y=1 or y=0) applies FIRST, before picking a formula? 3. Am I using p directly for the y=1 case, and 1-p for the y=0 case -- never mixing the two? 4. Have I remembered the leading -1 is applied LAST, after simplifying inside the brackets, not forgotten or added only at the end? 5. When a loss value comes out surprisingly large, have I checked whether the prediction was a 'confident wrong' case (pushing the relevant probability near 0)?